Inference in Propositional Logic and Introduction to First-order Logic

Giulia Manara

SDU

07/04/2026

Outline

1. Last time... - chapter 7
2. Resolution Algorithm for PL
3. Effective Propositional Model-checking
4. First-Order Logic - chapter 8
   - Using FOL

Last time... - chapter 7

Propositional Logic

Syntax of propositional logic:
We fix a set of propositional symbols $\mathbb{A}$, and we defines formulas as follows:

$$A, B ::= p \mid \neg A \mid A \wedge B \mid A \vee B \mid A \rightarrow B \mid A \leftrightarrow B$$

Semantics of propositional logic:
We define the semantics of propositional logic by:

- giving a truth value for each propositional symbol in $\mathbb{A}$ in a model $\mathfrak{M}$
- giving a truth table for each connective

A	$\neg A$
$\top$	$\bot$
$\top$	$\bot$
$\bot$	$\top$
$\bot$	$\top$

A	B	$A \wedge B$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\bot$
$\bot$	$\bot$	$\bot$

A	B	$A \vee B$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\top$
$\bot$	$\top$	$\top$
$\bot$	$\bot$	$\bot$

A	B	$A \rightarrow B$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\top$
$\bot$	$\bot$	$\top$

Models in Propositional Logic

A **model** $\mathfrak{M}$ is a function that assigns a truth value to each propositional variable in the language.

$$\mathfrak{M} \ : \ \mathbb{A} \to \{\bot, \top\}$$

For example, you can have two models $\mathfrak{M}_1$ and $\mathfrak{M}_2$ such that:

$\mathfrak{M}_1$	$\mathfrak{M}_2$
$p_1 \mapsto \top$	$p_1 \mapsto \bot$
$p_2 \mapsto \bot$	$p_2 \mapsto \top$
$p_3 \mapsto \top$	$p_3 \mapsto \top$
$\dots$	$\dots$

So $\mathfrak{M}_1(p_1) = \top$ while $\mathfrak{M}_2(p_1) = \bot$ and $\mathfrak{M}_1(p_3) = \mathfrak{M}_2(p_3) = \top$.

Entailment in Propositional Logic

Definition:

A sentence α ***entails*** a sentence β if every model that satisfies α also satisfies β

$$\alpha \models \beta \text{ iff } (M \models \alpha \text{ implies } M \models \beta \text{ for every model } M)$$

Two sentences α and β are ***logically equivalent*** if $\alpha \models \beta$ and $\beta \models \alpha$
$\alpha \equiv \beta$ iff $(\alpha \models \beta$ and $\beta \models \alpha)$

For example. . .

- $A \models A$.
- $A \models A \vee B$: if $\mathfrak{M} \models A$ then $\mathfrak{M} \models A \vee B$.
- $p \not\models p \wedge q$, with $p \neq q$: consider $\mathfrak{M}$ such that $\mathfrak{M}(p) = \top$ and $\mathfrak{M}(q) = \bot$, then $\mathfrak{M} \models p$ but $\mathfrak{M} \not\models p \wedge q$.

Logical Equivalences

$$A \rightarrow B \equiv \neg A \vee B$$

A	B	$A \rightarrow B$	$\neg A \vee B$
$\top$	$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$	$\bot$
$\bot$	$\top$	$\top$	$\top$
$\bot$	$\bot$	$\top$	$\top$

Logical Equivalences

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \rightarrow \beta) \equiv (\neg\beta \rightarrow \neg\alpha)$	contraposition
$(\alpha \rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \leftrightarrow \beta) \equiv ((\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Proof by contradiction

Theorem:

For any sentences α and β,

$$\alpha \models \beta \quad \text{iff} \quad \not\models \alpha \wedge \neg\beta$$

Inference Algorithms

How to check if $\alpha \models \beta$?

An ***inference algorithm*** is a procedure that takes as input a sentence α and a sentence β and returns 'yes' if $\alpha \models \beta$ and 'no' otherwise.

$$\alpha \vdash_{\text{MyAlgo}} \beta$$

An inference algorithm is

- ***sound*** (or ***truth-preserving***) if it only returns 'yes' when $\alpha \models \beta$.
- ***complete*** if it returns 'yes' every $\alpha \models \beta$.

Resolution Algorithm for PL

Resolution Rule

Do you remember *modus-ponens*?

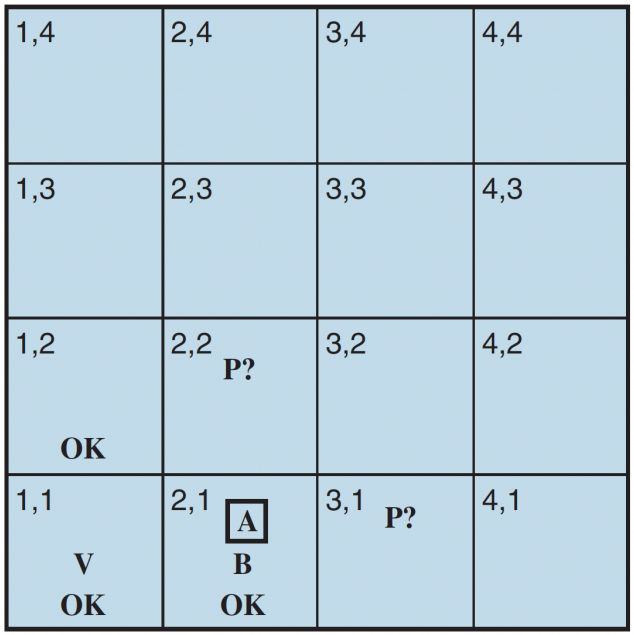
$$MP\ \frac{\alpha \rightarrow \beta \quad \alpha}{\beta} \text{ or equivalentely } MP\ \frac{\neg\alpha \vee \beta \quad \alpha}{\beta}$$

Resolution algorithm uses a generalization of modus ponens called **resolution rule**:

$$RR\ \frac{l_1 \vee \cdots \vee l_k \quad m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \ldots l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \cdots \vee m_n}$$

with $l_i = \neg m_j$, and l_x, l_y are literals, (propositional variables or their negation).

Key idea: resolution rule eliminates one complementary pair of literals (one positive and one negative) from two clauses to produce a new clause.



We know :

- $B_{2,1} \rightarrow P_{3,1} \vee P_{2,2} \vee P_{1,1} \equiv \neg B_{2,1} \vee P_{3,1} \vee P_{2,2} \vee P_{1,1}$
- $\neg P_{1,1}$ (there is no pit in $[1,1]$)
- $B_{2,1}$ (there is a breeze in $[2,1]$)

$$RR\ \frac{RR\ \dfrac{\neg B_{2,1} \vee P_{3,1} \vee P_{2,2} \vee P_{1,1} \quad B_{2,1}}{P_{3,1} \vee P_{2,2} \vee P_{1,1}} \qquad \neg P_{1,1}}{P_{3,1} \vee P_{2,2}}$$

Example: deriving the empty clause

Applying resolution can lead to derive the empty clause $\square$:

$$RR\ \frac{A \quad \neg A}{\square}$$

Assuming A and $\neg A$ both true is a **contradiction**

i.e. $A \wedge \neg A$ is **unsatisfiable**
(there is no $\mathfrak{M}$ such that $\mathfrak{M} \models A$ and $\mathfrak{M} \models \neg A$).

Why RR does not eliminate more than one pair of literals at a time?

Consider:

$$(p \vee q), \quad (\neg p \vee \neg q)$$

If we eliminate both pairs ($p, \neg p$ and $q, \neg q$), we get:

$$\square \quad \text{(empty clause)}$$

But this is wrong!

The clauses are satisfiable, e.g.:

$$\mathfrak{M} \text{ such that } \mathfrak{M}(p) = \text{true}, \quad \mathfrak{M}(q) = \text{false}$$

Key idea:

- Eliminating one pair preserves correctness (soundness)
- Eliminating multiple pairs may create **false contradictions**

Conjunctive Normal Form

The resolution algorithm works on sentences in conjunctive normal form (CNF):

$$\underbrace{(l_{1,1} \vee \cdots \vee l_{1,m_1})}_{\text{Clause 1}} \wedge \underbrace{(l_{2,1} \vee \cdots \vee l_{2,m_2})}_{\text{Clause 2}} \wedge \ \cdots \ \wedge \underbrace{(l_{n,1} \vee \cdots \vee l_{n,m_n})}_{\text{Clause n}}$$

where each clause is a disjunction of literals.

Example:

$$\underbrace{(p \vee \neg q)}_{\text{Clause 1}} \wedge \underbrace{(q \vee r \vee s)}_{\text{Clause 2}} \wedge \underbrace{(\neg p \vee \neg s)}_{\text{Clause 3}}$$

Fact: any propositional sentence can be transformed into an equivalent CNF.

Conjunctive Normal Form

Original formula:
$B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$

1. Eliminate $\leftrightarrow$ (replace $\alpha \leftrightarrow \beta$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$):
$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$

2. Eliminate $\rightarrow$ (replace $\alpha \rightarrow \beta$ with $\neg\alpha \vee \beta$):
$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$

3. Move $\neg$ inwards (De Morgan & double negation):
$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$

4. Apply distributivity ($\vee$ over $\wedge$) and flatten:
$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

Resolution Algorithm

Idea: to prove $KB \models \alpha$ show that $KB \wedge \neg\alpha$ is unsatisfiable, **proof by contradiction**.

The algorithm proceeds as follows:

- Convert $(KB \wedge \neg\alpha)$ into CNF.
- Apply resolution to derive new clauses.
- Repeat until:
  - **No new clauses** $\Rightarrow$ no contradiction found $\Rightarrow$ $KB \not\models \alpha$
  - **Empty clause** $\square$ **derived** $\Rightarrow$ contradiction
    $\Rightarrow$ $(KB \wedge \neg\alpha)$ is unsatisfiable
    $\Rightarrow$ $KB \models \alpha$

Resolution Algorithm

function PL-RESOLUTION(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  clauses $\leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
  new $\leftarrow \{\}$
  while *true* **do**
    for each pair of clauses C_i, C_j **in** *clauses* **do**
      resolvents $\leftarrow$ PL-RESOLVE(C_i, C_j)
      if *resolvents* contains the empty clause **then return** *true*
      new $\leftarrow$ *new* $\cup$ *resolvents*
    if *new* $\subseteq$ *clauses* **then return** *false*
    clauses $\leftarrow$ *clauses* $\cup$ *new*

Resolution is **sound** and **complete** for propositional logic.

Resolution Closure and Termination

Resolution closure:

$$RC(S) = \text{set of all clauses obtained from } S \text{ by repeated resolution}$$

Idea:

- Start from S
- Keep adding all possible **resolvents**
- Stop when no new clauses can be generated

Termination:

- Only finitely many clauses can be built from the symbols in S
- $\Rightarrow$ $RC(S)$ cannot grow forever

$\Rightarrow$ Resolution always terminates

Completeness of Resolution (Proof Idea)

Goal:

$$S \text{ unsatisfiable} \Rightarrow \square \in RC(S)$$

Proof idea (contrapositive):

$$\square \notin RC(S) \Rightarrow S \text{ satisfiable}$$

Main argument:

- Consider the closure $RC(S)$
- If $\square \notin RC(S) \Rightarrow$ no contradiction is derived
- $\Rightarrow$ we can assign truth values without falsifying clauses
- $\Rightarrow$ we can build a **model**

$$\Rightarrow S \text{ is satisfiable}$$

Example: Resolution Algorithm

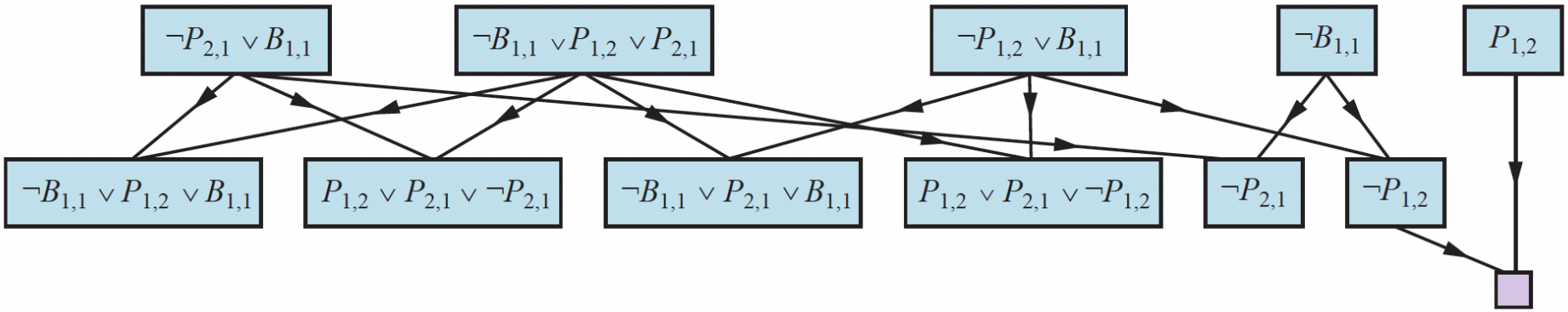


Figure 7.14 Partial application of PL-RESOLUTION to a simple inference in the wumpus world to prove the query $\neg P_{1,2}$. Each of the leftmost four clauses in the top row is paired with each of the other three, and the resolution rule is applied to yield the clauses on the bottom row. We see that the third and fourth clauses on the top row combine to yield the clause $\neg P_{1,2}$, which is then resolved with $P_{1,2}$ to yield the empty clause, meaning that the query is proven.

Effective Propositional Model-checking

Effective Propositional Model-checking

- We consider **DPLL algorithm**: for solving the SAT problem.
- **SAT problem:** Given a set of clauses $C_1, C_2, \ldots, C_n$, determine whether there exists a model $\mathfrak{M}$ where all clauses are true.
- based on **model checking**: test whether a formula is true in a model.
- Why it is important:
  - Many problems in computer science can be turned into SAT.
  - Better SAT algorithms help solve bigger, harder problems.

DPLL Algorithm

- DPLL solves SAT by recursively exploring possible models (depth-first search).
- Input: a formula in **CNF** (a set of clauses).
- Tricks that make it faster:
  - **Early termination:** Detects whether the sentence must be true or false, even with a partially completed model.
  - **Pure symbol:** If a variable always appears with the same sign, assign it to make its clauses true.
  - **Unit clause:** If a clause has only one unassigned variable, assign it to satisfy the clause. Repeat if this creates new unit clauses (unit propagation).
- These tricks avoid checking all possibilities and speed up the search.

Optimizations tricks

- **Component analysis:** Solve independent sets of clauses separately to speed up search.
- **Variable and value ordering:** Pick the most frequent variable first; try true before false.
- **Intelligent backtracking:** Not just chronological backtracking: back up to the relevant point of conflict and remember conflicts to avoid repeating them.
- **Random restarts:** Restart the search with different random choices, a run appears stuck; keep learned clauses.
- **Clever indexing:** Keep track of which variables appear in which clauses to quickly update as assignments are made.

First-Order Logic - chapter 8

Pros and Cons of Propositional Logic

Pros

- **Declarative**: knowledge and inference are separate, inference is domain-independent
- Allows partial, disjunctive, or negated information
- **Compositional**: meaning of $A \wedge B$ comes from A and B
- **Context-independent**: inference is not affected by the context

Cons

- Limited expressive power
  Example: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

First-Order Logic

In propositional logic, the world contains facts.

In **first-order logic**, the world contains:

- **Objects:** people, houses, numbers, ...
- **Relations:**
  - Unary: red, round, prime, ...
  - Binary: brother of, bigger than, occurred after, ...
- **Functions:** owned by, one more than, ...

FOL Syntax

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \\
&\mid \neg\, \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
&\mid \textit{Quantifier Variable}, \ldots \textit{Sentence} \\
\textit{Term} &\rightarrow \textit{Function}(\textit{Term}, \ldots) \\
&\mid \textit{Constant} \\
&\mid \textit{Variable} \\
\textit{Quantifier} &\rightarrow \forall \mid \exists \\
\textit{Constant} &\rightarrow A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} &\rightarrow a \mid x \mid s \mid \cdots \\
\textit{Predicate} &\rightarrow \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} &\rightarrow \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Terms – Objects

Terms refer to objects in the world.

- **Constant:** a specific object Example: KingJohn
- **Function(term,...):** specify an object using an expression Example: LeftLegOf(KingJohn)
- **Variable:** a generic object Example: x, y

Atomic Sentences – Facts

Atomic sentences are the basic facts about objects.

- **Zero-ary predicates:** like propositional variables
- n-**ary predicates:** Predicate$(t_1, \ldots, t_n)$
- **Equality:** $\mathrm{t}_1 = \mathrm{t}_2$

Examples:

- Brother(KingJohn, RichardTheLionheart)
- $>$ (Length(LeftLegOf(Richard)), Length(LeftLegOf(KingJohn)))
- $2 + 2 = 4$

Complex Sentences – Complex Facts

Complex sentences are built from atomic sentences using logical connectives:

$$\neg S, \quad S_1 \wedge S_2, \quad S_1 \vee S_2, \quad S_1 \Rightarrow S_2, \quad S_1 \Leftrightarrow S_2$$

Examples:

- Sibling(KingJohn, Richard) $\Rightarrow$ Sibling(Richard, KingJohn)
- $> (1,2) \vee \leq (1,2)$
- $> (1,2) \wedge \neg > (1,2)$

Quantifiers in First-Order Logic

Universal Quantifier – "For all"

$$\forall x \, \text{King}(x) \Rightarrow \text{Person}(x)$$

The sentence $\forall x \, P$ states that P is true for **every object** x.

Existential Quantifier – "There exists"

$$\exists x \, \text{Crown}(x) \wedge \text{OnHead}(x, \mathit{John})$$

The sentence $\exists x \, P$ states that P is true for **at least one object** x.

First-Order Logic – Example Sentences

- Brothers are siblings:

$$\forall x, y \ \text{Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

- "Sibling" is symmetric:

$$\forall x, y \ \text{Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$$

- One's mother is one's female parent:

$$\forall x, y \ \text{Mother}(x, y) \Leftrightarrow (\text{Female}(x) \wedge \text{Parent}(x, y))$$

- A first cousin is a child of a parent's sibling:

$$\forall x, y \ \text{FirstCousin}(x, y) \Leftrightarrow$$

$$\exists p, ps(\text{Parent}(p, x) \wedge \text{Sibling}(ps, p) \wedge \text{Parent}(ps, y))$$

First-Order Logic: Models

A model $\mathfrak{M}$ defines the meaning of a first-order language:

$$\mathfrak{M} = (\mathbb{D}, \mathcal{I})$$

- **Domain:** $\mathbb{D}$, a set of objects, $\mathbb{D} \neq \emptyset$
- **Interpretation:** $\mathcal{I}$, assigns meaning to symbols
  - Constants $\rightarrow$ objects in $\mathbb{D}$
  - Predicate symbols $\rightarrow$ relations over $\mathbb{D}$
  - Function symbols $\rightarrow$ functional relations over $\mathbb{D}$

First-Order Logic: Truth of Atomic Sentences

Atomic sentence:

$$P(t_1, t_2, \ldots, t_n)$$

is **true in a model $\mathfrak{M}$ iff** the objects denoted by $t_1, \ldots, t_n$ are in the relation assigned to P by the interpretation $\mathcal{I}$:

$$\mathfrak{M} \models P(t_1, \ldots, t_n) \quad \text{iff} \quad (t_1^{\mathfrak{M}}, \ldots, t_n^{\mathfrak{M}}) \in \mathcal{I}(P)$$

Key idea: truth is defined with respect to a **model** that gives meaning to constants, functions, and predicates.

First-Order Logic: Truth of Complex Sentences

Truth of connectives is **exactly as in propositional logic**.

Universal quantifier:

$$\mathfrak{M} \models \forall x\, P(x) \quad \text{iff} \quad \mathfrak{M} \models P(d) \text{ for every } d \in \mathbb{D}$$

Existential quantifier:

$$\mathfrak{M} \models \exists x\, P(x) \quad \text{iff} \quad \mathfrak{M} \models P(d) \text{ for at least one } d \in \mathbb{D}$$

Key idea: quantifiers **range over the model's domain** and extend propositional reasoning to first-order logic.

First-Order Logic: example of a model

Domain (Objects):

$$\mathbb{D} = \mathbb{N} \quad \text{(natural numbers)}$$

Signature of the language:

$$\Sigma = \{\underbrace{0, 1, 2, \ldots,}_{\text{Constants}} \quad \underbrace{+}_{\text{Function}} , \quad \underbrace{>}_{\text{Predicate}} \}$$

Atomic sentences:

$$0 + 1 > 0, \quad 2 + 3 > 4$$

Key idea: symbols define the "vocabulary" but have no meaning yet.

First-Order Logic: example of a model

Model:

$$\mathfrak{M} = (\mathbb{D}, \mathcal{I})$$

Interpretation $\mathcal{I}^{\mathfrak{M}}$:

$$0^{\mathfrak{M}} = 0, \quad 1^{\mathfrak{M}} = 1, \quad 2^{\mathfrak{M}} = 2$$

$$+^{\mathfrak{M}} = \text{usual addition over } \mathbb{N}$$

$$>^{\mathfrak{M}} = \text{usual "greater than" relation on } \mathbb{N}$$

Example evaluation:

$$\mathfrak{M} \models 2 + 3 > 4 \quad \text{(true)}$$

Key idea: the model $\mathfrak{M}$ gives concrete meaning to symbols.

Truth in First-Order Logic: example of a model

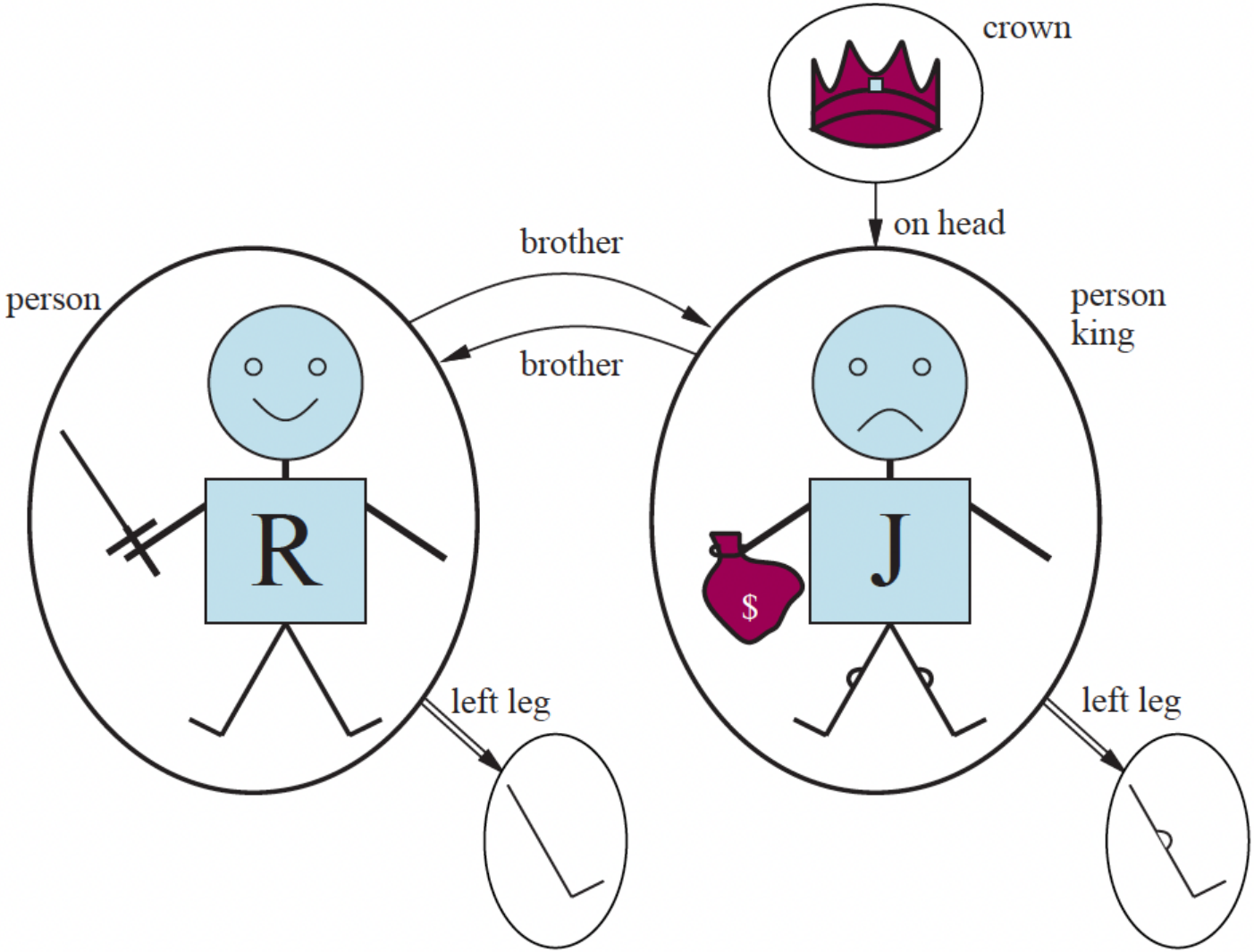


Figure 8.2 A model containing five objects, two binary relations (brother and on-head), three unary relations (person, king, and crown), and one unary function (left-leg).

Natural Numbers – Peano Axioms

Signature of the language:

$\Sigma = \{\text{Constants: } 0, \quad \text{Function: } S(\cdot) \text{ (successor)}, \quad \text{Predicate: } \text{NatNum}(\cdot)\}$

Peano Axioms:

$$\text{NatNum}(0) \quad \text{and} \quad \forall n \, \text{NatNum}(n) \Rightarrow \text{NatNum}(S(n))$$

Successor constraints:

$$\forall n \, 0 \neq S(n), \quad \forall m, n \; m \neq n \Rightarrow S(m) \neq S(n)$$

Key idea: Starting from 0 and applying S, we generate all natural numbers:

$$0, S(0), S(S(0)), \ldots$$

Defining Addition in FOL

Addition in terms of successor:

$$\forall m\ \text{NatNum}(m) \Rightarrow +(0, m) = m$$

$$\forall m, n\ \text{NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow +(S(m), n) = S(+(m, n))$$

Infix notation (syntactic sugar):

$$S(n) \equiv n + 1, \quad (m + 1) + n = (m + n) + 1$$

Key idea: Addition is repeated application of the successor function.